

Fundamentals of programming & Data Management Project 2025/26 - Mod B



UNIVERSITY OF NAPLES FEDERICO II
Master Degree in Data Science

Voice2Query: AI-Powered Speech-to-SQL for Interactive Database Exploration

Professor: Vincenzo Moscato

Group Members:

1. Shreya Jadhav Mukunda Rao – D03000276
2. Venkata Vamsi Krishna Cheticom – D03000285
3. Shashank Hubballi Lokesh – D03000286
4. Ajay Kumar Tatikonda – D03000287

Abstract

Voice2Query is an AI-powered Speech-to-SQL system designed to simplify database interaction through natural voice commands. The system integrates Automatic Speech Recognition (ASR), Natural Language Processing (NLP), Text-to-SQL generation, and data visualization into a unified pipeline. Spoken queries are first transcribed into text using Whisper, then translated into SQL statements using Ollama with Llama 3. The generated SQL queries are executed on a SQLite database, and the results are displayed through an interactive Streamlit dashboard. By eliminating the need for manual SQL writing, the system enables intuitive and accessible database exploration for users with little or no database expertise. The project demonstrates the practical integration of speech recognition, large language models, and relational database technologies in a fully functional end-to-end application.

Contents

1. Introduction

1.1 Project objective

2. System design & Architecture

2.1 Architecture Diagram

2.2 Technologies Used

2.3 Database Schema Design

2.4 Working Functionality of the System

3 Results

Conclusion

1. Introduction

- In recent years, the growing volume of structured data has increased the importance of database management systems in various domains, including education, business, healthcare, and entertainment. Relational databases are widely used for storing and managing structured information; however, interacting with these databases typically requires knowledge of Structured Query Language (SQL). This requirement can be a significant barrier for non-technical users who need to access and analyze data but lack database expertise.
- Advances in Artificial Intelligence (AI), Natural Language Processing (NLP), and Speech Recognition technologies have created new opportunities for developing intuitive interfaces that allow users to communicate with computer systems using natural language. Speech-based interfaces, in particular, offer a more accessible and user-friendly way of interacting with databases by enabling users to ask questions verbally rather than writing complex SQL queries.
- This project, titled **Voice2Query: AI-Powered Speech-to-SQL for Interactive Database Exploration**, aims to bridge the gap between natural human communication and relational database systems. The proposed system enables users to query a database through voice commands. The spoken input is first converted into text using Automatic Speech Recognition (ASR), then translated into SQL queries using a Large Language Model (LLM), executed on a relational database, and finally presented through an interactive dashboard.
- The system integrates multiple technologies, including Whisper for speech recognition, Ollama with Llama 3 for Text-to-SQL generation, SQLite for database management, and Streamlit for visualization and user interaction. By combining these components into a unified pipeline, the project demonstrates how modern AI techniques can simplify database access and improve usability for users without SQL knowledge.
- The primary objective of this project is to design and implement a complete end-to-end Speech-to-SQL system capable of transforming spoken language into executable SQL queries and displaying meaningful results in a clear and interactive manner. The project also explores the practical challenges involved in speech recognition, natural language understanding, SQL generation, and system integration while evaluating the effectiveness of the overall solution.

1.1 Project objective

The primary objective of this project is to develop a complete Speech-to-SQL system that enables users to interact with a relational database using natural spoken language instead of manually writing SQL queries. The system aims to provide an intuitive and accessible interface for database exploration, particularly for users with limited knowledge of database query languages.

The specific objectives of the project are as follows:

1. **Design and Implement a Relational Database**
 - Create and manage a structured relational database containing meaningful data.
 - Define an appropriate schema and populate the database with sample records.
2. **Implement Speech Recognition**
 - Convert user voice commands into textual form using Automatic Speech Recognition (ASR) technology.
 - Evaluate available ASR solutions and select a suitable tool based on accuracy, deployment requirements, and performance.
3. **Develop a Text-to-SQL Translation Module**
 - Transform natural language queries into valid SQL statements.
 - Utilize a Large Language Model (LLM) to understand user intent and generate accurate SQL queries based on the database schema.
4. **Execute SQL Queries on the Database**
 - Process generated SQL statements and execute them on the relational database.
 - Retrieve and manage query results efficiently.
5. **Create an Interactive Dashboard**
 - Develop a user-friendly interface that integrates all system components.
 - Display transcribed speech, generated SQL queries, and query results in an intuitive manner.
6. **Integrate an End-to-End Speech-to-SQL Pipeline**
 - Combine speech recognition, natural language processing, SQL generation, database execution, and visualization into a unified system.
 - Ensure smooth communication between all modules.
7. **Evaluate System Performance**
 - Assess the effectiveness of speech recognition and SQL generation.
 - Analyze the system's usability, accuracy, and overall functionality.

By achieving these objectives, the project demonstrates how Artificial Intelligence, Natural Language Processing, and Database Management technologies can be integrated to create an efficient voice-driven database querying system that simplifies access to structured data.

2. System Design and Architecture

The Voice2Query system follows a cascaded Speech-to-SQL architecture that transforms spoken user queries into executable SQL statements and presents the results through an interactive dashboard. The system is composed of five major components: Speech Input, Speech Recognition, Text-to-SQL Translation, Database Execution, and Visualization.

The workflow begins when the user records a voice query through the Streamlit dashboard. The recorded audio is passed to the Speech Recognition module, which uses the Whisper model to convert speech into text. This transcribed text represents the user's natural language query.

The generated text is then forwarded to the Text-to-SQL module. This module utilizes Ollama with the Llama 3 large language model to interpret the user's intent and generate an appropriate SQL query based on the database schema. Prompt engineering techniques are applied to ensure that the model returns syntactically correct SQL statements without additional explanations.

Once the SQL query is generated, it is sent to the SQL Execution module. This component connects to the SQLite database, executes the query, and retrieves the requested data. The database contains structured movie information, including attributes such as movie title, release year, rating, and genre.

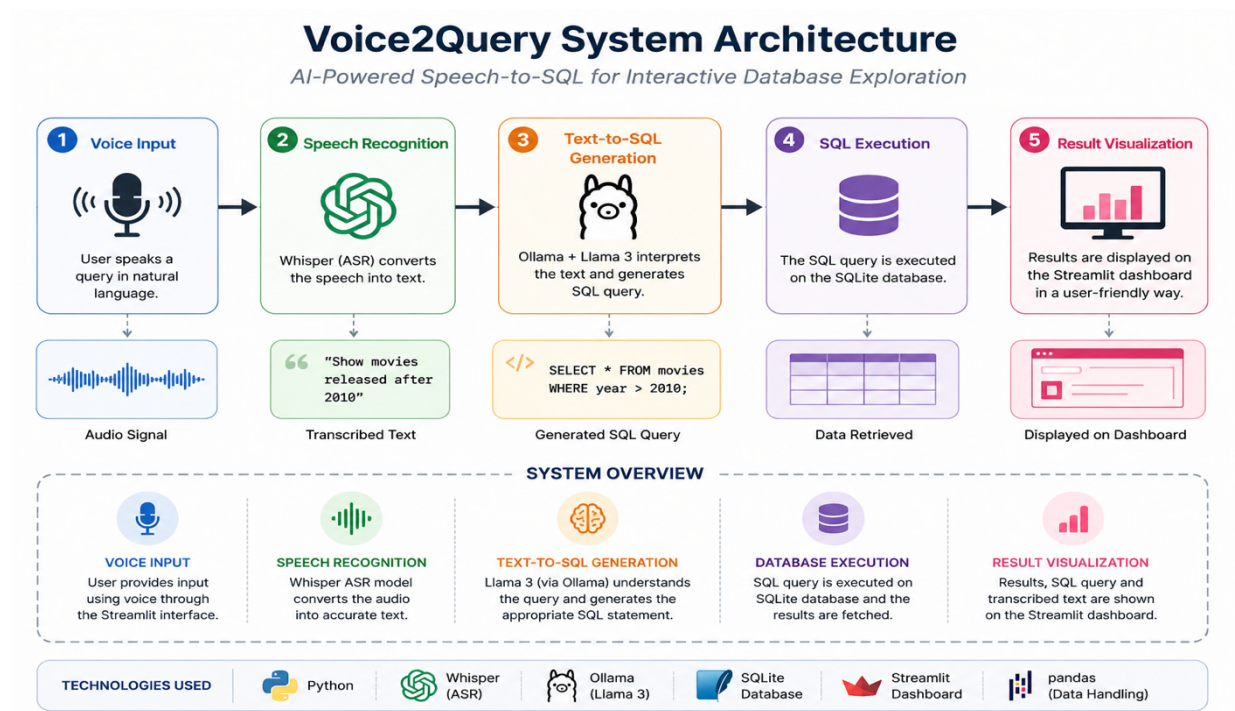
Finally, the query results are displayed through the Streamlit dashboard. The dashboard presents the recognized speech, generated SQL query, and query results in a user-friendly format, enabling users to easily explore database contents without writing SQL commands manually.

The overall architecture can be represented as follows:

- ⇒ User Speech
 - Audio Recording
 - Whisper Speech Recognition
 - Natural Language Text
 - Ollama + Llama 3 Text-to-SQL Translation
 - SQL Query Generation
 - SQLite Database Execution
 - Streamlit Dashboard Visualization

This modular architecture provides flexibility, ease of maintenance, and the ability to independently improve individual components such as speech recognition, SQL generation, or visualization without affecting the entire system.

2.1 Architecture Diagram



2.2 Technologies Used

The Voice2Query system integrates multiple technologies to achieve end-to-end Speech-to-SQL functionality. Each technology was selected based on its performance, ease of integration, cost-effectiveness, and suitability for local deployment. The following sections describe the role of each component in the system.

1. Python

Python was used as the primary programming language for implementing the entire application. It provides extensive support for Artificial Intelligence, Machine Learning, Database Management, and Web Development through its rich ecosystem of libraries and frameworks. Python's simplicity and flexibility enabled rapid development and seamless integration of all project components.

2. Whisper (Speech Recognition)

Whisper is an Automatic Speech Recognition (ASR) model developed by OpenAI. It is capable of converting spoken language into text with high accuracy and supports multiple languages. Whisper was selected because it can run locally without requiring cloud services or API subscriptions.

- Converts user voice commands into text
- Acts as the Speech-to-Text component of the pipeline

Workflow

1. User records audio through the dashboard.
2. Audio is saved as a WAV file.
3. Whisper processes the audio.
4. Transcribed text is generated.

Why whisper is need in this project?

- High transcription accuracy
- Multilingual support
- Offline execution
- No usage cost
- Easy integration with Python

Whisper provides an effective balance between accuracy and computational requirements while eliminating dependency on external cloud services.

3. Ollama

Ollama is a local Large Language Model (LLM) serving platform that allows AI models to run directly on a user's machine. Instead of relying on cloud APIs, Ollama hosts and manages models locally through a lightweight server.

- Provides a local environment for running Llama 3
- Receives natural language input
- Returns generated SQL queries

Importance of Ollama

- Fully offline operation
- No API costs
- Improved privacy and security
- Easy deployment
- Supports multiple open-source models

The initial implementation considered cloud-based AI APIs, but Ollama was chosen because it eliminates recurring costs and enables local deployment.

4. Llama 3 (Text-to-SQL Generation)

Llama 3 is a Large Language Model used for natural language understanding and SQL query generation. In this project, it serves as the core Text-to-SQL engine.

- Receives transcribed user queries
- Understands user intent
- Generates corresponding SQL statements

Example

Natural Language Query:

"Show movies released after 2010"

Generated SQL:

SELECT * FROM movies WHERE year > 2010;

- Strong language understanding capabilities
- Supports complex query generation
- Flexible prompt-based interaction
- Works locally through Ollama

Llama 3 provides high-quality SQL generation while remaining freely available for local deployment.

5. SQLite

SQLite is a lightweight relational database management system used to store and manage structured movie data. It operates as a file-based database and does not require a separate server installation.

- Stores movie information
- Executes generated SQL queries
- Returns query results

Database Schema

Movies Table:

- ⇒ id
- ⇒ title
- ⇒ year
- ⇒ rating
- ⇒ genre

SQLite offers simplicity, portability, and sufficient functionality for developing and testing the Speech-to-SQL pipeline.

6. Streamlit

Streamlit is an open-source framework used for building interactive web applications directly in Python. It was used to create the user interface for the Voice2Query system.

- Provides the dashboard interface
- Allows voice recording interaction
- Displays transcribed speech
- Shows generated SQL queries

- Visualizes query results

Streamlit enables rapid development of professional dashboards while maintaining a simple Python-based workflow.

7. Pandas

Pandas is a data analysis library used for processing and displaying query results.

- Reads SQL query results
- Converts results into dataframes
- Displays structured tables within Streamlit

Component	Technology Used	Purpose
Programming Language	Python	Application Development
Speech Recognition	Whisper	Speech-to-Text Conversion
LLM Runtime	Ollama	Local Model Serving
Text-to-SQL Model	Llama 3	SQL Query Generation
Database	SQLite	Data Storage and Query Execution
Dashboard	Streamlit	User Interface and Visualization
Data Processing	Pandas	Query Result Management

Together, these technologies form a complete end-to-end Speech-to-SQL system capable of transforming spoken language into executable SQL queries and presenting results through an intuitive dashboard interface.

2.3 Database Schema Design

Overview

A relational movie database was designed and implemented to serve as the data source for the Voice2Query system. The database stores information about movies and allows users to retrieve data through voice-generated SQL queries. The schema was intentionally designed to be simple yet sufficiently rich to support a variety of SQL operations such as filtering, sorting, aggregation, and conditional queries.

The database was implemented using SQLite and contains a single table named **movies**. The table stores essential movie attributes, including movie identification, title, release year, rating, and genre.

Movies Table Schema

The movies table is designed as follows:

Attribute	Data Type	Description
id	INTEGER	Unique identifier for each movie (Primary Key)
title	TEXT	Name of the movie
year	INTEGER	Release year of the movie
rating	REAL	Movie rating on a scale from 1 to 5
genre	TEXT	Category or genre of the movie

SQL Table Definition

```
CREATE TABLE movies (  
  id INTEGER PRIMARY KEY,  
  title TEXT,  
  year INTEGER,  
  rating REAL,  
  genre TEXT  
);
```

Entity Description

Movie Entity

The Movie entity represents information related to a film. Each movie record contains:

- A unique movie identifier.
- The movie title.
- The year of release.
- The movie rating.
- The movie genre.

Example records:

id	title	year	rating	genre
1	Inception	2010	4.8	Sci-Fi
2	Interstellar	2014	4.7	Sci-Fi
3	Titanic	1997	4.5	Romance
4	The Dark Knight	2008	4.9	Action
5	Avatar	2009	4.4	Sci-Fi

The database contains multiple movie records covering different genres and release years to support diverse query scenarios.

Schema Design Rationale

The schema was designed to support the objectives of the Speech-to-SQL system. The selected attributes allow users to perform meaningful database queries such as:

- Retrieve movies released in a specific year.
- Find movies with ratings above a threshold.
- Filter movies by genre.
- Display movies released after a particular year.
- Sort movies based on rating or release date.

Example user queries include:

- "Show all movies released after 2010."
- "List movies with rating greater than 4.5."
- "Display all Sci-Fi movies."
- "Show movies released in 2014."

These natural language queries are translated into SQL statements and executed on the movie database.

Entity-Relationship Representation

Since the current implementation uses a single table, the schema consists of one entity:

MOVIES	

id (PK)	
title	
year	
rating	
genre	

2.4 Working Functionality of the System

The Voice2Query system implements a complete end-to-end Speech-to-SQL pipeline that enables users to interact with a relational database using natural spoken language. The system integrates speech recognition, natural language processing, SQL generation, database execution, and result visualization into a unified workflow.

1. Voice Input Module

The process begins when the user provides a voice input through the Streamlit dashboard. The user clicks the “Start Recording” button and speaks a natural language query, such as:

- “Show all movies released after 2010”
- “List movies with rating above 4.5”

The system records the audio and stores it as a WAV file for further processing.

2. Speech-to-Text Conversion

The recorded audio is processed using the Whisper Automatic Speech Recognition (ASR) model. Whisper converts the spoken language into accurate textual form.

Example:

Input (Voice):

“Show movies released in 2010”

Output (Text):

Show movies released in 2010

This step ensures that speech input is transformed into machine-readable text.

3. Text-to-SQL Generation

The transcribed text is passed to the Text-to-SQL module, which uses Ollama with the Llama 3 large language model. The model interprets the natural language query and generates a corresponding SQL statement based on the predefined database schema.

Example:

Input (Text):

Show movies released in 2010

Output (SQL):

```
SELECT * FROM movies WHERE year = 2010;
```

Prompt engineering techniques are used to ensure that the model produces only valid SQL queries without additional explanations.

4. SQL Query Execution

The generated SQL query is executed on the SQLite database. The database contains structured movie records, including title, release year, rating, and genre.

The system processes the SQL query and retrieves the matching results from the database.

Example Result:

id	title	year	rating	genre
1	Inception	2010	4.8	Sci-Fi

5. Result Visualization

The final output is displayed on a Streamlit dashboard. The interface shows:

- The transcribed speech
- The generated SQL query
- The query results in tabular format

The dashboard provides a clear and interactive visualization of the entire pipeline, making it easy for users to understand the transformation from speech to data retrieval.

Example Use Case

User Input (Voice):

“Show me movies with rating above 4.7”

Generated SQL:

```
SELECT * FROM movies WHERE rating > 4.7;
```

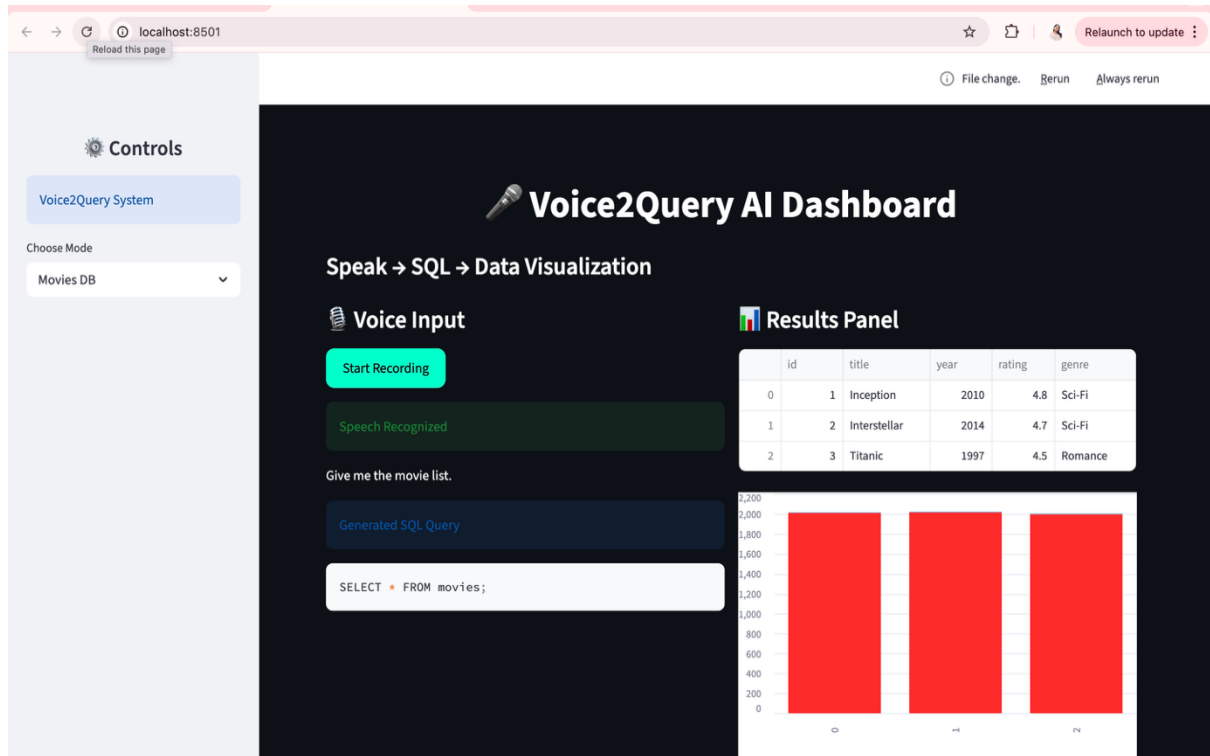
Output:

The system returns a list of high-rated movies from the database.

3. Results

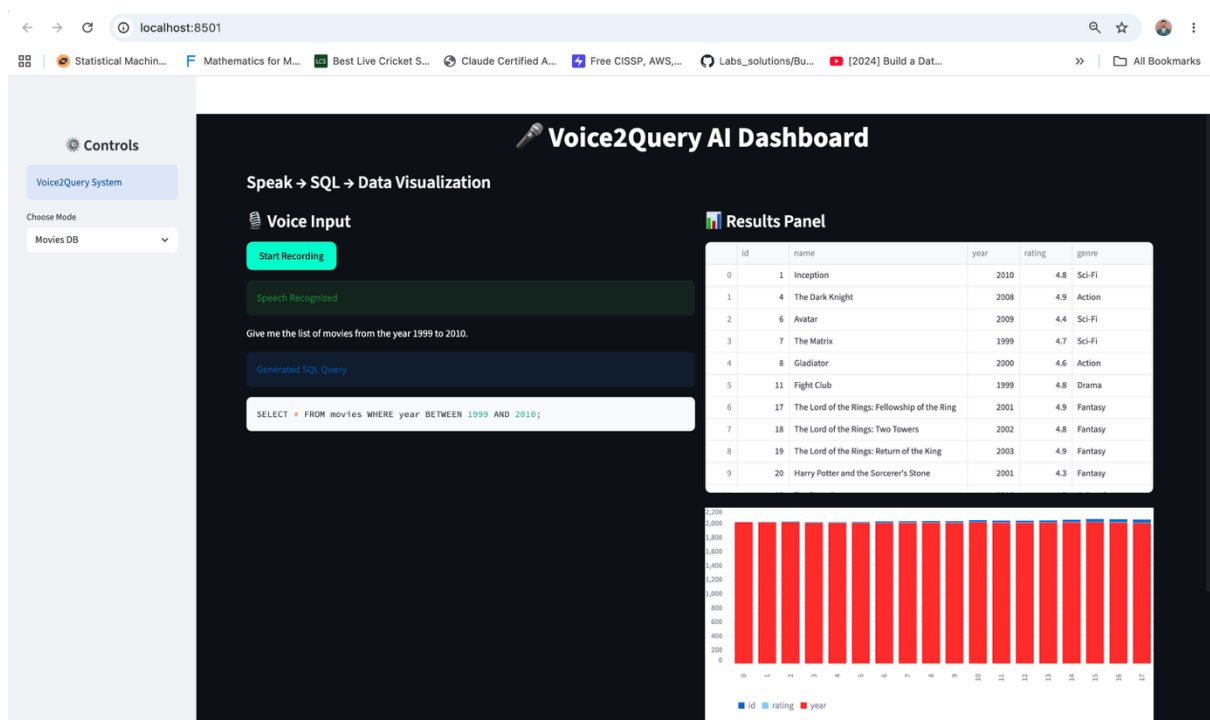
Voice: Give me the movie list

Sql: `SELECT * FROM movies;`

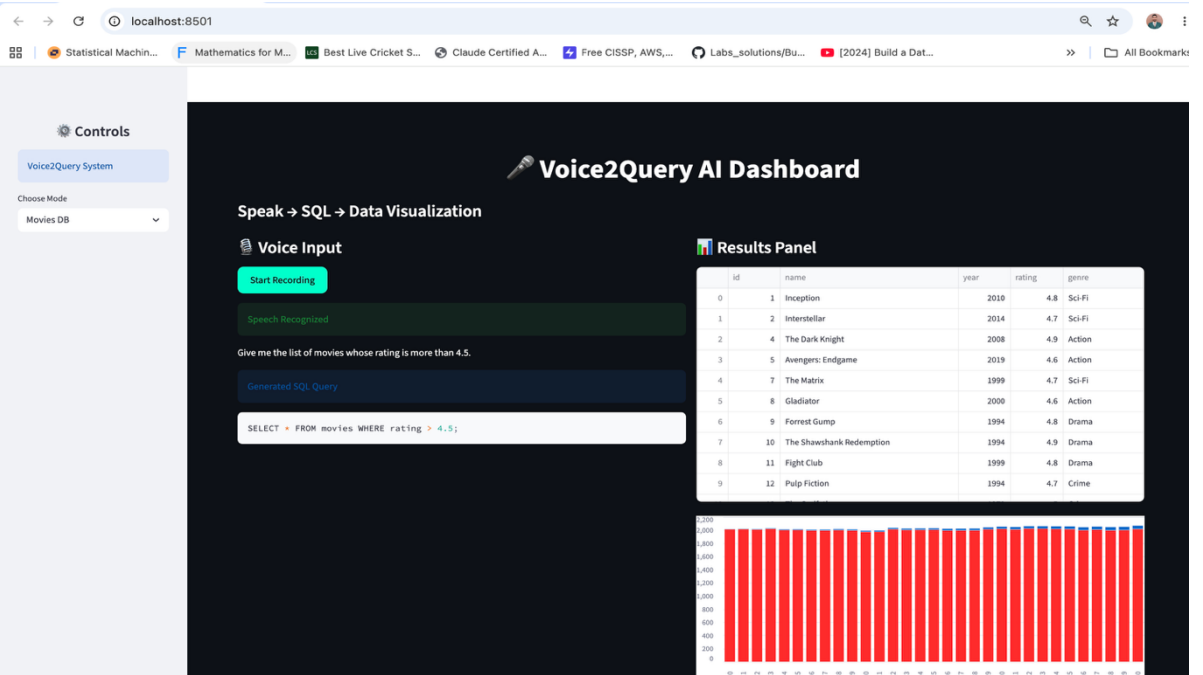


Voice: Give me the list of movies from the year 1999 to 2010

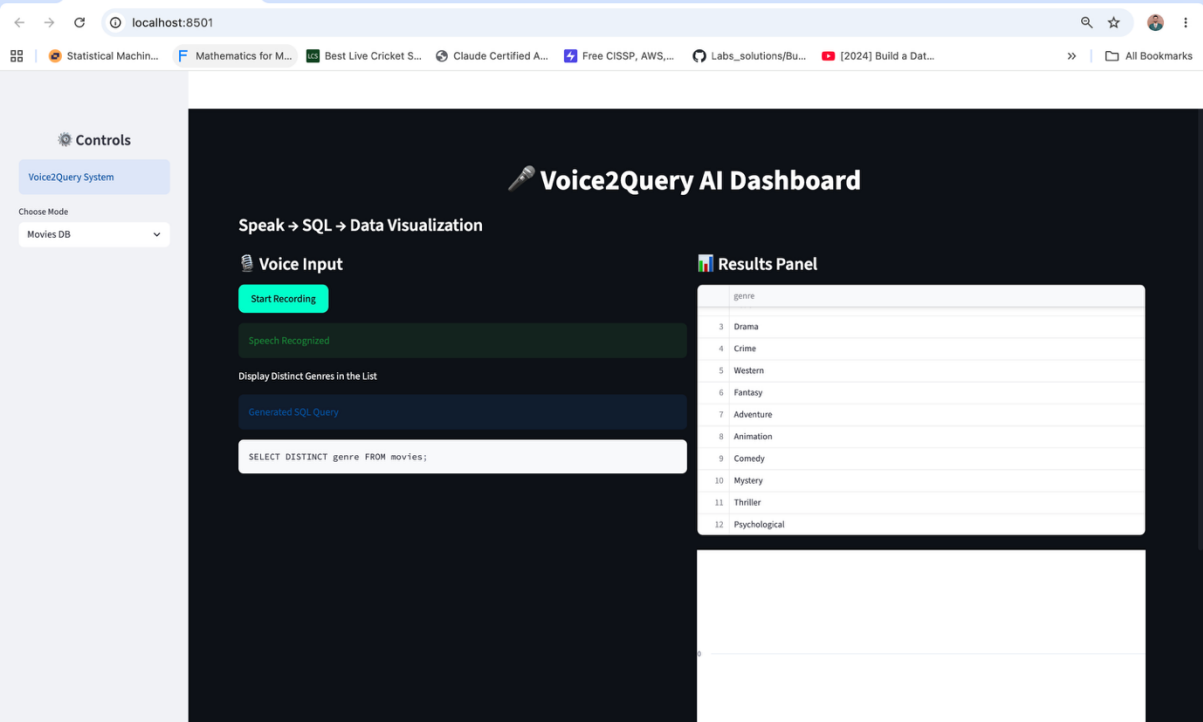
Sql: `SELECT * FROM movies WHERE year BETWEEN 1999 AND 2010;`



Voice: Give me the list of movies ratings more than 4.5
Sql: SELECT * FROM movies WHERE rating > 4.5;

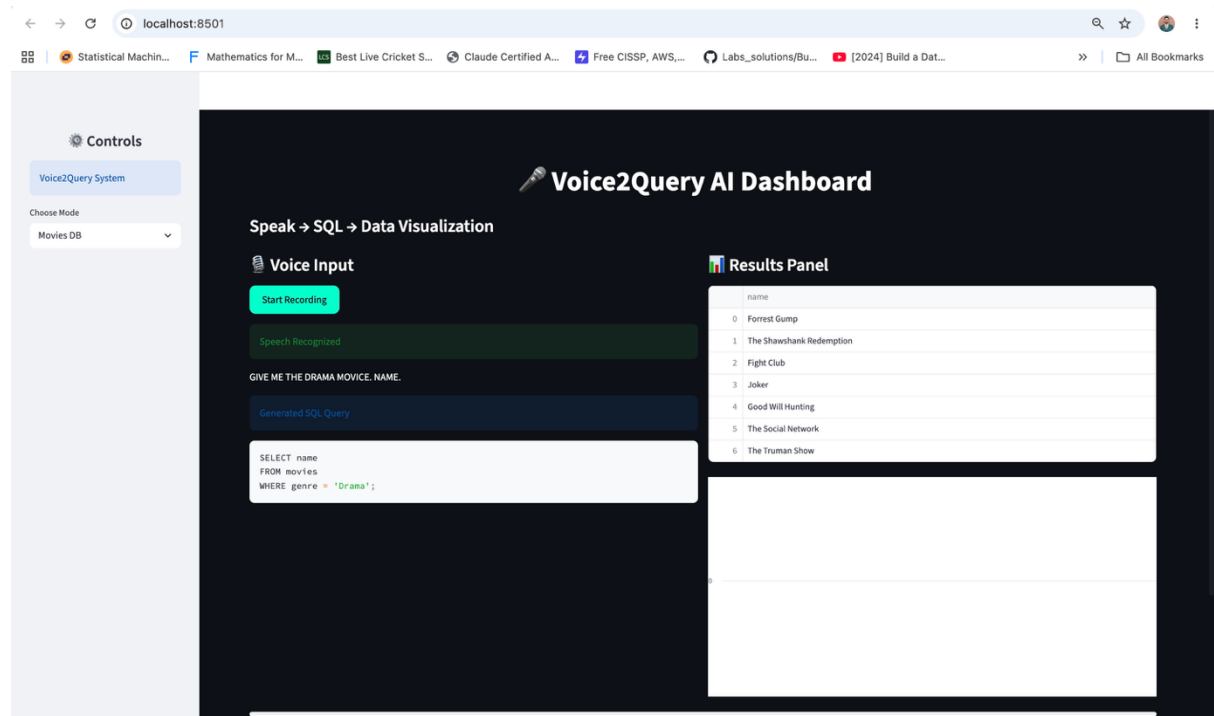


Voice: Display distinct genres in the list
Sql: SELECT DISTINCT genre FROM movies;



Voice: Give me the drama movie names

Sql: `SELECT name
FROM movies
WHERE genre = 'drama';`



Conclusion

Voice2Query provides an end-to-end Speech-to-SQL solution that enables intuitive voice-based interaction with databases. By combining Whisper, Ollama with Llama 3, SQLite, and Streamlit, the system successfully converts spoken language into SQL queries, executes them on a relational database, and visualizes the results through an interactive dashboard. The project demonstrates how AI and data management technologies can be integrated to create a practical and user-friendly database exploration system.